Fundamental Algorithm Techniques – Problem Set #10

Problem 1 — P, NP, NP-complete, NP-hard

Definitions (short & simple)

- **P**: problems solvable in **polynomial time**
- **NP**: solutions can be **verified** in polynomial time
- **NP-complete**: hardest problems **inside NP**
- **NP-hard**: at least as hard as NP-complete, may not be in NP
- **Undecidable**: cannot be solved by any algorithm

1. find max, linear search, shortest path in unweighted graph, matrix multiplication

All have **polynomial-time algorithms**.

Class: P

2. sorting of list, Dijkstra (non-negative), BFS, DFS, merge sort, quicksort

All are classic **efficient algorithms**.

Class: P

3. Sudoku

- Checking a solution → polynomial
- Finding a solution → hard

Class: NP-complete

4. 3-coloring of graph, scheduling with conflicts

Both are classic NP-complete problems.

Class: NP-complete

5. Traveling Salesperson Problem, Hamiltonian Cycle, Clique

All are famous hard problems in NP.

Class: NP-complete

6. Cryptography, factoring large integers

- Verification is easy
- No known polynomial-time solution

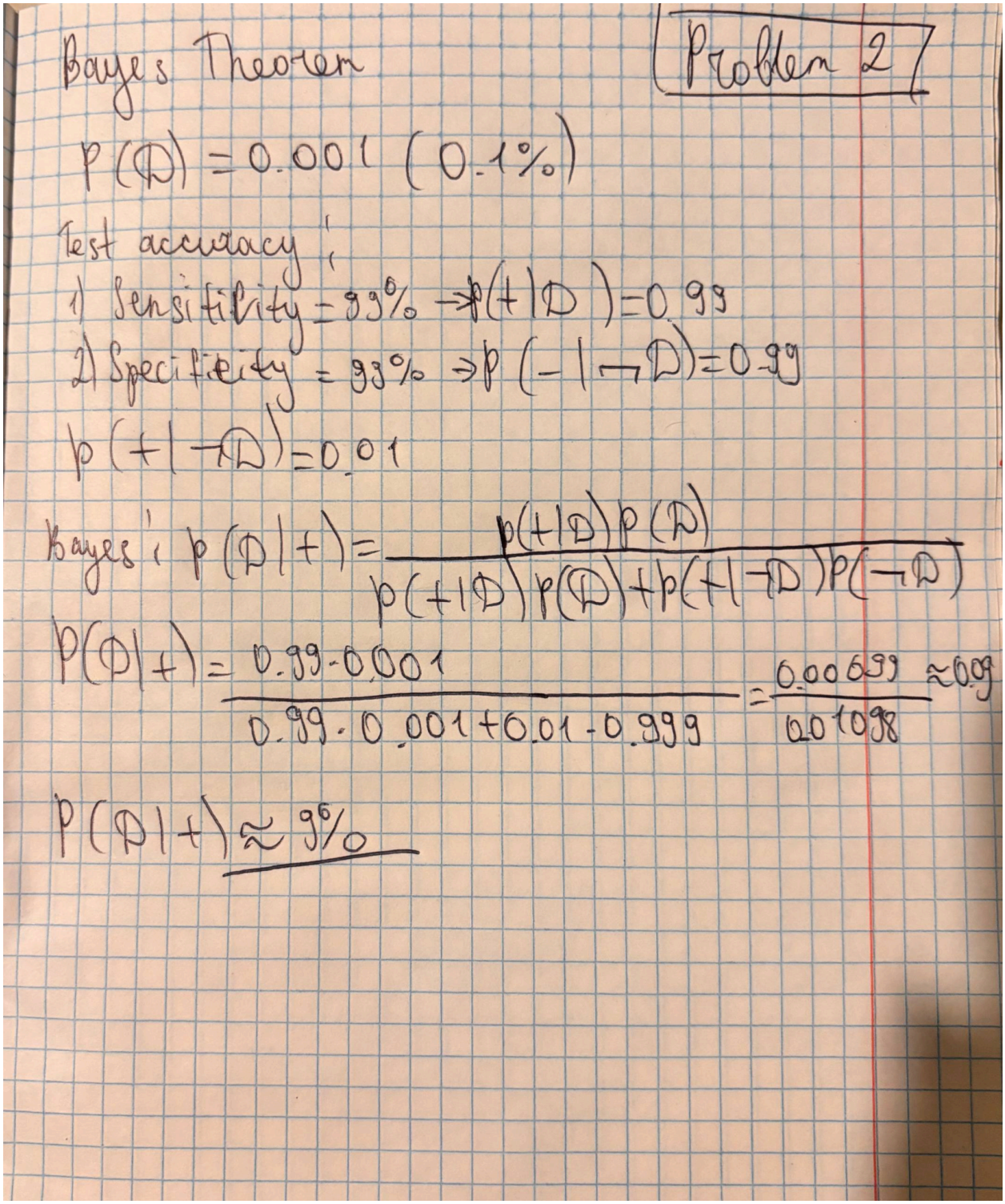
Class: NP (not known to be P or NP-complete)

7. Halting Problem, Busy Beaver

No algorithm can solve these in general.

Class: Undecidable

Problem 2 — Bayes Theorem



Explanation:
Even accurate tests give many false positives when the disease is rare.

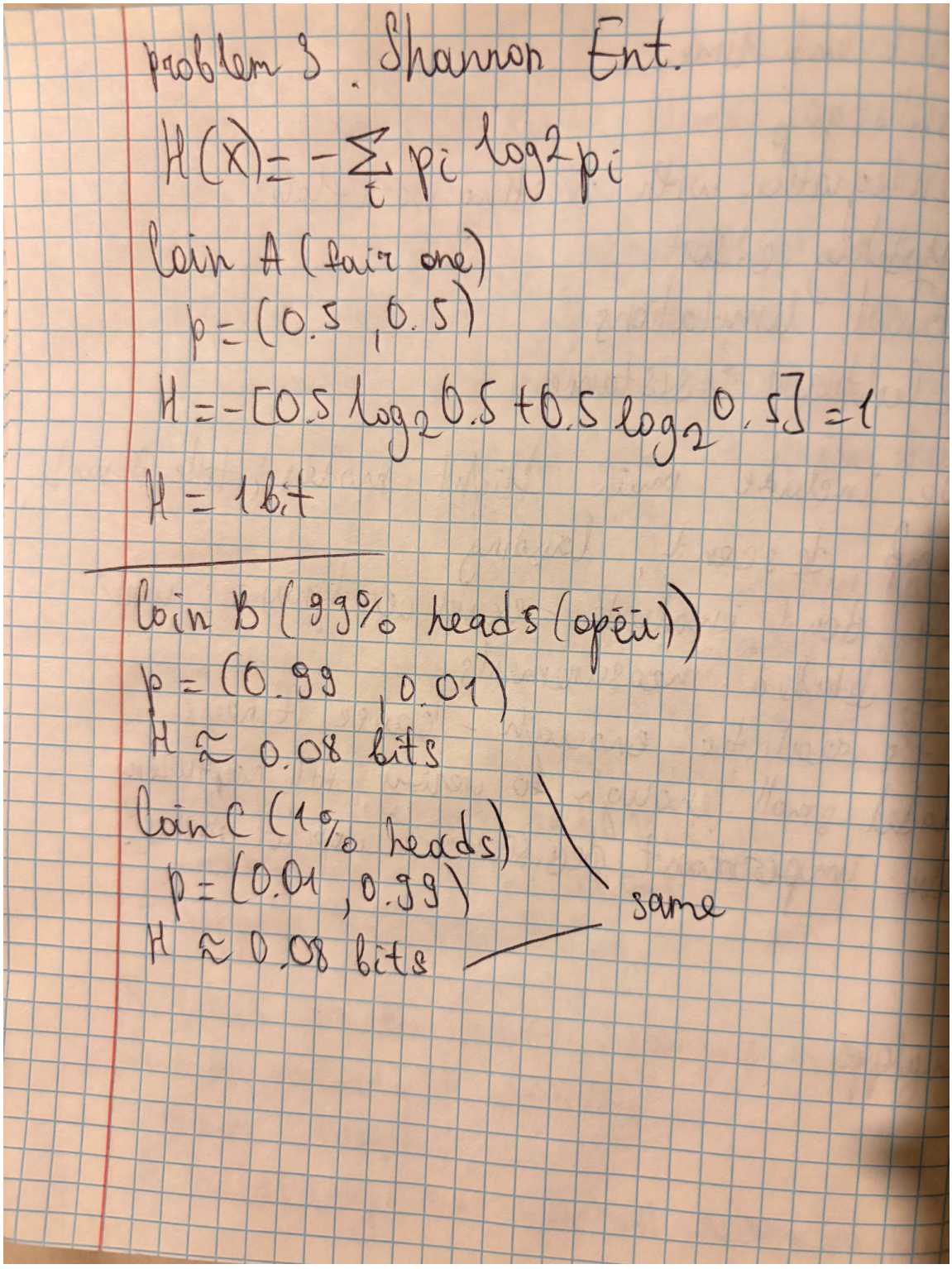
Optional Python code

```
P_D = 0.001
P_not_D = 1 - P_D
P_pos_given_D = 0.99
P_pos_given_not_D = 0.01

P_D_given_pos = (P_pos_given_D * P_D) / (
    P_pos_given_D * P_D + P_pos_given_not_D * P_not_D
)

print("Probability patient has disease:", round(P_D_given_pos * 100, 2), "%")
```

Problem 3 — Shannon Entropy



Explanation

- Fair coin → maximum uncertainty → **1 bit**
- Biased coin → predictable → **few bits**

- Rare events → **huge surprise**, but **low average entropy**

Python code

```
import math

def entropy(p):
    return -sum(pi * math.log2(pi) for pi in p if pi > 0)

coins = {
    "Fair coin": [0.5, 0.5],
    "99% heads": [0.99, 0.01],
    "1% heads": [0.01, 0.99]
}

for name, p in coins.items():
    print(name, "Entropy =", round(entropy(p), 3), "bits")
```